Distributed Database

Database II - Lecture 7

1. Introduction

Distributed database provides a number of advantages of distributed computing to the DBMS domain. A distributed computing system consists of a number of processing elements that are interconnected by a computer network, and that cooperate in performing certain application tasks. The distributed database is a collection of multiple logically interrelated databases distributed over a computer network.

Parallel processing divides a complex task into many smaller tasks, and executes the smaller tasks simultaneously in several tasks. Thus, the complex task is completed with better performance and also quickly. The parallel database system makes use of the parallelism in DBMS and achieves high performance and high availability database servers at much lower price.

A distributed database is a collection of data which belong logically to the same system but are spread over the sites of a computer network. This definition emphasizes two equally important aspects of a distributed database as follows:

1. Distribution. The fact that the data are not resident at the same site (processor), so that we can distinguish a distributed database from a single, centralized database.
2. Logical correlation. The fact that the data have some properties which tie them together, so that we can distinguish a distributed database from a set of local databases or files which are resident at different sites of a computer network.

A distributed database is a collection of data which are distributed over different computers of a computer network. Each site of the network has autonomous processing capability and can perform local applications. Each site also participates in the execution of at least one global application, which requires accessing data at several sites using a communication subsystem.
A simple distributed database on a local network is shown in Fig. 1.

An important property of distributed database management systems (DDBMSs) is whether they are homogeneous or heterogeneous. Homogeneous DDBMS refers to a DDBMS with the same DBMS at each site, even if the computers and/or the operating systems are not the same. A heterogeneous DDBMS uses instead at least two different DBMSs. Heterogeneous DDBMS adds the problem of translating between the different data models of the different local DBMSs to the complexity of homogeneous DDBMSs.

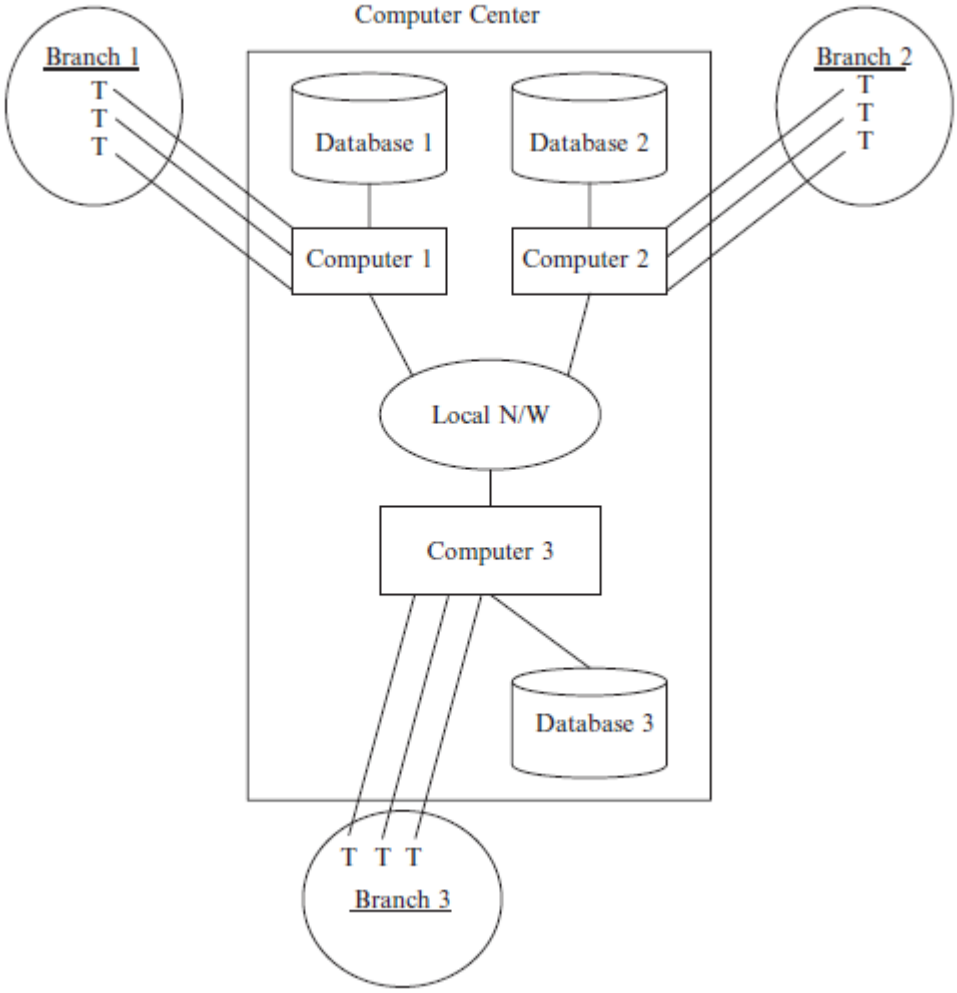

Fig. 1. A distributed database on a local network

2. Architectural Models for Distributed DBMS

Let us consider the possible ways in which multiple databases may be put together for sharing by multiple DBMSs.We use a classification that organizes the systems as characterized with respect to (1) the autonomy of local systems, (2) their distribution, and (3) their heterogeneity.

Autonomy

It refers to the distribution of control, not of data. It indicates the degree to which individual DBMS can operate independently. Autonomy is a function of a number of factors such as whether they can independently execute transactions, and whether one is allowed to modify them. Requirements of an autonomous system have been specified in a variety of ways.

Distribution

There are a number of ways DBMSs can be distributed. We abstract the alternatives into two classes: client/server distribution and peer-to-peer distribution. The client/server distribution concentrates data management duties at servers while the clients focus on providing the application environment including the user interface. The communication duties are shared between the client machines and servers. Client/server DBMSs represent the first attempt at distributing functionality. There are a variety of ways of structuring them, each providing a different level of distribution. In

peer-to-peer systems, there is no distinction of client machines vs. servers. Each machine has full DBMS functionality and can communicate with other machines to execute queries and transactions.

Heterogeneity

It may occur in various forms in distributed systems, ranging from hardware heterogeneity and differences in networking protocols to variations in data managers. Heterogeneity in query languages not only involves the use of completely different data access paradigms in different data models, but also covers difference in languages even when the individual systems use the same data model. Different query languages that use the same data model often select very different methods for expressing identical requests.

3. Types of Distributed DBMS Architecture

The distributed DBMS architecture types namely client server and peer-to-peer systems are discussed below.

3.1 Client/Server Systems

The general idea of this architecture is simple and elegant: distinguish the functionality that needs to be provided and divide these functions into two classes as server functions and client functions. This provides a two level architecture which makes it easier to manage the complexity of modern DBMSs and the complexity of distribution.

As with any highly popular term, client/server has been much abused and has come to mean different things. If one takes a process-centric view, then any process that requests the services of another process is its client and vice versa. In this sense, “client/server computing” and “client/server DBMS,” as it is used in its more modern context, do not refer to processes, but to actual machines.

The architecture shown in Fig. 2 is quite common in relational systems where the communication between the clients and the server without trying to understand or optimize them. The server does most of the work and returns the result relation to the client. There are a number of different types of client/server architecture. The simplest is the case where there is only one server which can be accessed by multiple clients. We call this as “multiple client–single server.” From a data management perspective, this is not much different from centralized databases since the database is stored on only one machine (the server) which also hosts the software to manage it.

There are two management strategies in multiple client–multiple server: either each client manages its own connection to the appropriate server or each client knows of only its “home server” which then communicates with the other servers as required. The former approach simplifies server code, but loads the client machines with additional responsibilities. This leads to what has been called as “heavy client” systems. The latter approach, on the other hand, concentrates the data management functionality at the servers. Thus, the transparency of data access is provided at the server interface, leading to “light clients.”

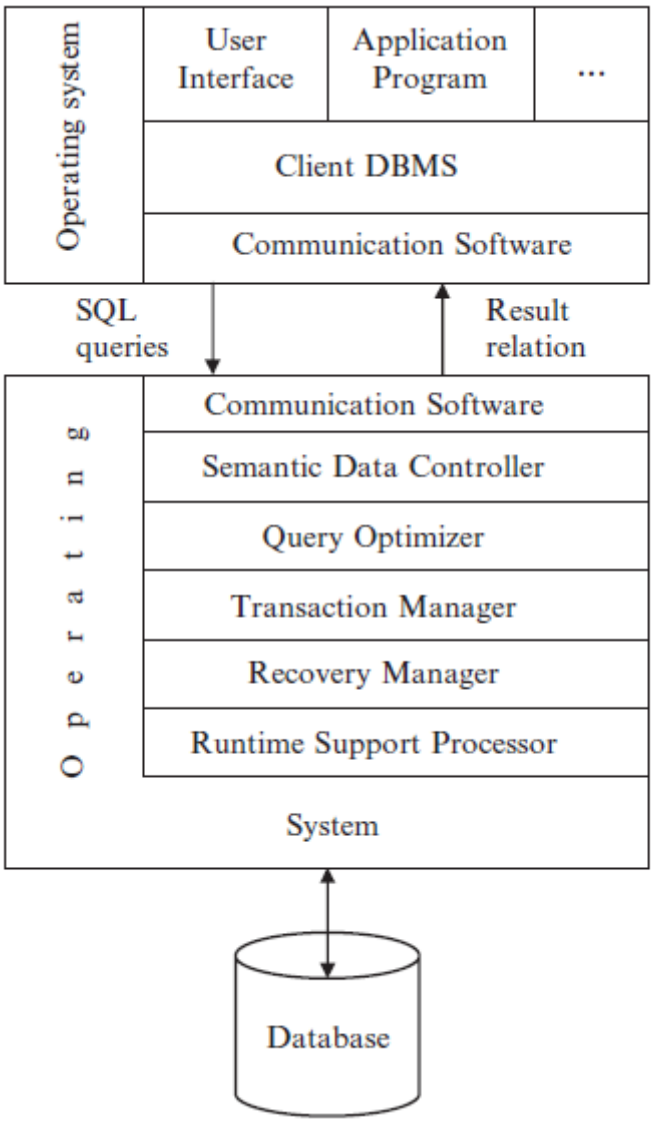

Figure. 2. Client/server reference architecture

3.2 Peer-to-Peer Distributed Systems

The architecture of this model provides the levels of transparency. Data independence is supported since the model is an extension of ANSI/SPARC, which provides such independence naturally. Network transparency, on the other hand, is supported by the definition of the global conceptual schema. The user queries data irrespective of its location or of which local component of the distributed database system will service it.

3.2 Distributed Data Storage

Consider a relation r that is to be stored in the database. There are two approaches to storing this relation in the distributed database:

• Replication. The system maintains several identical replicas (copies) of the relation, and stores each replica at a different site. The alternative to replication is to store only one copy of relation r.

• Fragmentation. The system partitions the relation into several fragments, and stores each fragment at a different site.

Fragmentation and replication can be combined: A relation can be partitioned into several fragments and there may be several replicas of each fragment. In the following subsections, we elaborate on each of these techniques.

3.2.1 Data Replication

If relation r is replicated, a copy of relation r is stored in two or more sites. In the most extreme case, we have full replication, in which a copy is stored in every site in the system.

There are a number of advantages and disadvantages to replication.

- Availability. If one of the sites containing relation r fails, then the relation r can be found in another site. Thus, the system can continue to process queries involving r, despite the failure of one site.

- Increased parallelism. In the case where the majority of accesses to the relation r result in only the reading of the relation, then several sites can process queries involving r in parallel. The more replicas of r there are, the greater the chance that the needed data will be found in the site where the transaction is executing. Hence, data replication minimizes movement of data between sites.

- Increased overhead on update. The system must ensure that all replicas of a relation r are consistent; otherwise, erroneous computations may result. Thus, whenever r is updated, the update must be propagated to all sites containing replicas. The result is increased overhead. For example, in a banking system, where account information is replicated in various sites, it is necessary to ensure that the balance in a particular account agrees in all sites.

In general, replication enhances the performance of read operations and increases the availability of data to read-only transactions. However, update transactions incur greater overhead. Controlling concurrent updates by several transactions to replicated data is more complex than in centralized systems, which we studied in the previous lecture. We can simplify the management of replicas of relation r by choosing one of them as the **primary copy** of r.

3.2.2 Data Fragmentation

If relation r is fragmented, r is divided into a number of fragments r1, r2, . . . , rn. These fragments contain sufficient information to allow reconstruction of the original relation r. There are two different schemes for fragmenting a relation: ***horizontal***

fragmentation and ***vertical fragmentation***. Horizontal fragmentation splits the relation by assigning each tuple of r to one or more fragments. Vertical fragmentation splits the relation by decomposing the scheme R of relation r.

In **horizontal fragmentation**, a relation r is partitioned into a number of subsets. Each tuple of relation r must belong to at least one of the fragments, so that the original relation can be reconstructed, if needed.

As an illustration, the account relation can be divided into several different fragments, each of which consists of tuples of accounts belonging to a particular branch. Horizontal fragmentation is usually used to keep tuples at the sites where they are used the most, to minimize data transfer.

In its simplest form, **vertical fragmentation** is the same as decomposition. Vertical fragmentation of r(R) involves the definition of several subsets of attributes R1, R2, . . . , Rn of the schema R. The fragmentation should be done in such a way that we can reconstruct relation r from the fragments by taking the natural join.

One way of ensuring that the relation r can be reconstructed is to include the primary-key attributes of R in each Ri. More generally, any superkey can be used. It is often convenient to add a special attribute, called a tuple-id, to the schema R. The tuple-id value of a tuple is a unique value that distinguishes the tuple from all other tuples. The tuple-id attribute thus serves as a candidate key for the augmented schema, and is included in each Ri.

To illustrate vertical fragmentation, consider a university database with a relation employee info that stores, for each employee, employee id, name, designation, and salary. For privacy reasons, this relation may be fragmented into a relation employee private info containing employee id and salary, and another relation employee public info containing attributes employee id, name, and designation. These may be stored at different sites, again, possibly for security reasons.

The two types of fragmentation can be applied to a single schema; for instance, the fragments obtained by horizontally fragmenting a relation can be further partitioned vertically. Fragments can also be replicated. In general, a fragment can be replicated, replicas of fragments can be fragmented further, and so on.